PER%SENSE — GO IMPLEMENTATION

Actuarial & Life-Contingency Implementation Walkthrough

How the Go port values life-contingent cash flows: the recovered 1988 mortality tables, the life-table mathematics, the contingency combinations, the payment-on-death valuation, and how they weave into present value — now validated against the real DOS program to the cent.

Scope. The Go package `internal/finance/actuarial/` — `table.go` (the `LifeTable` and its survival mathematics), `contingency.go` (the contingency combinator and payment-on-death valuation), and `persense1988.go` (the recovered original 1988 HHS tables) — plus its integration into present value (`presentvalue/calc.go`, `backward.go`, `variablerate.go`) and the API/frontend wiring.

Audience. Engineers maintaining or extending the Go port. Every claim is anchored to a `file:line` reference under `internal/`.

Lineage & status. The original DOS `ACTUARY` unit *source* is still missing (its `LifeProb`/`PODValue` bodies are absent — see the companion `legacy/code_docs/actuarial_dos_walkthrough.pdf`). But the two things that earlier made this engine a pure reconstruction have since been recovered: the original **mortality table data** (MALE.ACT / FEMALE.ACT, now embedded) and a runnable **DOS binary**. The Go engine running on the recovered tables now reproduces the real DOS program **to the cent** on a four-case golden grid (`docs/dos_actuarial_golden.md`). This is no longer "correct-by-construction only."

Contents

0 How to read this document

The life-contingency feature values annuities that continue "for life" and benefits paid "on death." It is an *overlay* on the present-value engine: each cash flow is weighted by the probability the contingency holds at its payment date, and a payment-on-death (POD) term is added to the total. Read the present-value Go walkthrough first.

0.1 Where the feature lives

Three files in `internal/finance/actuarial/` form the computational core, and a thin layer in `presentvalue/` wires it into the PV engine:

```
internal/finance/actuarial/
   |- table.go         LifeTable: SurvivalProb, ConditionalSurvival, LifeExpectancy, loaders
   |- contingency.go   ActuarialConfig, LifeProb (1- & 2-life), PODValue / PODValueFunc
   |- persense1988.go  RECOVERED 1988 HHS qx tables (Persense1988Male/Female)
internal/finance/presentvalue/
   |- calc.go          forward weighting + POD add + two-life guard + unknown-POD solve
   |- variablerate.go  same overlay under a piecewise rate schedule
   |- backward.go      peel POD, invert the weighted factor, prohibit date-solves
```

0.2 Citation convention

References look like `contingency.go:142` and resolve under `internal/finance/actuarial/` unless another path is named (e.g. `presentvalue/calc.go:498`). Pascal lineage of an integration point is noted as `PRESVALU.pas:313` — those numbers are recomputed against `legacy/src_documented/dos_source/` in the DOS companion document.

1 The config & types

1.1 ActuarialConfig

`ActuarialConfig` `contingency.go:86` carries everything the missing DOS unit needed: up to two life tables and dates of birth, the valuation date `Now` (the "alive at now" anchor — DOS `actu_now`), the POD benefit amount, and a `PODUnknown` flag.

```go
type ActuarialConfig struct {
    Table1 *LifeTable     // life table, person 1
    DOB1   types.DateRec  // date of birth, person 1
    Table2 *LifeTable     // optional second table (two-life contingencies)
    DOB2   types.DateRec
    Now    types.DateRec  // reference date — "alive at now" (DOS actu_now)
    POD    float64        // payment-on-death benefit
    PODUnknown bool        // solve for POD (DOS ComputeUnknownPOD); POD ignored as input
}
```

When `PODUnknown` is set, the engine back-solves the death benefit from a target Sum Value and the `POD` input is ignored.

1.2 Contingency constants

The seven contingency constants `contingency.go:13` mirror the surviving Pascal (`PETYPES.PAS:253`). `RequiresSecondLife` `contingency.go:28` flags the four that reference person 2 (`Only1Living`, `Only2Living`, `EitherLiving`, `BothLiving`). `ContingencyLabel` / `ContingencyFromCode` `contingency.go:38` / `62` map the `N L D 1 2 E B` letter codes (`PEDATA.pas:186`).

2 The recovered 1988 mortality tables

2.1 persense1988.go

This is the headline change since the previous revision of this document. The original DOS Per%Sense shipped two mortality tables — `MALE.ACT` and `FEMALE.ACT`, the 1988 U.S. Department of Health & Human Services qx tables. Those distribution files were recovered from a copy of the original software and parsed into Go arrays in `persense1988.go`:

- `Persense1988MaleQx` `persense1988.go:24` — age-indexed qx for ages 0..115 (closes at `qx[115]=1.0`).
- `Persense1988FemaleQx` `persense1988.go:46` — same range.
- `Persense1988Male()` / `Persense1988Female()` `persense1988.go:66` / `71` build a `LifeTable` from each via `NewLifeTableFromQx`.

The source files express each qx as "`x.xxxE-3`" (×0.001); the header of the sibling `88B&WM&F.ACT` distribution file names the basis as "USA Dept of HHS, 1988." Running the actuarial engine against these tables reproduces the original program's mortality assumptions exactly — unlike the SSA-2021 table the web build also offers.

> **Why this matters.** Earlier, the table *values* were unknown, so the engine could only be checked against textbook mathematics. With the real tables embedded, the engine can be — and now is (§6) — diffed against the real DOS program on its own mortality basis.

2.2 The female age-0 omission, preserved faithfully

The original `FEMALE.ACT` begins at age 1 — it has *no age-0 row*. The Go port preserves that exactly: `Persense1988FemaleQx[0] = 0` `persense1988.go:47`, with a flagged note `persense1988.go:16` rather than a silently invented value. This is immaterial to any real contingency (a reference age of 0 never occurs in practice — nobody is valued from birth) but is documented and carries a `// TODO: verify logic` marker about how the original engine read the missing row.

2.3 How tables reach the engine

The actuarial core is deliberately *data-driven*: it holds no hard-wired table. Tables flow in through two doors:

1. **API / PV layer.** The HTTP request carries `table1` / `table2` as `[[age, qx], …]` arrays; `buildActuarialConfig` `api/handlers.go:1547` parses each with `ParseJSON` and populates `ActuarialConfig.Table1` / `Table2`.
2. **Web frontend.** `cmd/persense/static/lifetables.js` ships four tables — `SSA_2021_MALE_QX`, `SSA_2021_FEMALE_QX`, `PERSENSE_1988_MALE_QX`, `PERSENSE_1988_FEMALE_QX` — and the actuarial panel exposes them in a dropdown (`index.html` `#actu-table1`). **SSA-2021 Male is the default selection**; the Per%Sense-1988 tables are selectable for DOS-faithful reproduction. A test (`persense1988_test.go: TestPersense1988MatchesServedJS`) guards against the Go arrays and the served JS drifting apart.

3 The life-table mathematics

A `LifeTable` `table.go:30` holds `Qx` (death probability within a year at each integer age) and `Lx` (survivors from a 100,000 radix). The two are inter-derivable; the survival functions read off `Lx`.

3.1 SurvivalProb & uniform-distribution-of-deaths interpolation

`SurvivalProb(age)` `table.go:44` returns the probability of surviving from birth to a (possibly fractional) age. For a non-integer age it linearly interpolates `lx` between the two bracketing integer ages — the standard **uniform-distribution-of-deaths (UDD)** assumption — then normalizes by `lx[0]`:

```
intAge = floor(age);  frac = age - intAge                          table.go:52-53
l(age)  = Lx[intAge]*(1 - frac) + Lx[intAge+1]*frac                table.go:66   (linear, UDD)
return    l(age) / Lx[0]                                           table.go:67
```

Boundary handling: `age <= 0` returns 1.0 (certain survival to birth); `age >= MaxAge` returns 0.0 (the table closes). Because ages enter as fractional years (a 65-and-a-half-year-old is `age = 65.5`), this interpolation is what lets the engine value a payment that falls between birthdays.

3.2 ConditionalSurvival — the quantity the engine actually uses

Cash flows are never weighted by survival-from-birth; they are weighted by survival *conditional on being alive at the valuation date*. `ConditionalSurvival(currentAge, futureAge)` `table.go:73` is the textbook `t·p·x`:

```
  t·p·x  =  l(futureAge) / l(currentAge)                           table.go:81-82
         =  SurvivalProb(futureAge) / SurvivalProb(currentAge)
```

It short-circuits to 1.0 when `futureAge <= currentAge` (no future shrinkage of the cohort before "now"), and to 0.0 if the cohort at `currentAge` has already vanished. Every per-person survival probability in §4 is a call to this function with `currentAge = age@Now`.

3.3 LifeExpectancy

`LifeExpectancy(age)` `table.go:88` sums forward conditional survival to the table's max age — the curtate expectation `e(x) = Σₜ t·p·x`. It is a convenience/reporting quantity; the PV overlay does not depend on it.

3.4 Loaders

`NewLifeTableFromQx` `table.go:104` builds `Lx` from `Qx` by the recurrence `lx[i+1] = lx[i]·(1 − qx[i])` off a 100,000 radix; `NewLifeTableFromLx` `table.go:119` inverts it (`qx[i] = 1 − lx[i+1]/lx[i]`, clamped to [0,1]). `ParseCSV` `table.go:139` and `ParseJSON` `table.go:211` read "`age,value`" rows from CSV or `[[age, qx], …]` from JSON; the API uses `ParseJSON`.

4 Contingency combination & payment-on-death

4.1 Per-person survival

`survivalProb1/2(date)` `contingency.go:114` / `125` each compute `ConditionalSurvival(age@Now, age@date)`, with ages measured by a 360-day `yearsDif` `contingency.go:103`.

> **The "immortal person 2" default.** `survivalProb2` returns `1.0` when no second table is set `contingency.go:126` — a silent default that makes a two-life formula collapse to a single life. This is harmless only because the two-life guard (§5.2) rejects any two-life contingency that lacks a usable second table *before* the math runs.

4.2 Two-life combination under independence

`LifeProb(date, contingency)` `contingency.go:142` combines one or two *independent* lives. Writing `s1`, `s2` for the two conditional survival probabilities, each contingency is a closed-form combination:

Contingency	LifeProb	Reading
`Living`	`s1`	person 1 alive
`Dead`	`1 − s1`	person 1 dead
`Only1Living`	`s1·(1 − s2)`	#1 alive AND #2 dead
`Only2Living`	`(1 − s1)·s2`	#2 alive AND #1 dead
`EitherLiving`	`1 − (1 − s1)(1 − s2)`	last-survivor: at least one alive
`BothLiving`	`s1·s2`	joint-life: both alive

The two structurally interesting combinations are the joint-life status (`BothLiving = s1·s2` — the product of independent survivals `contingency.go:162`) and the last-survivor status (`EitherLiving = 1 − (1 − s1)(1 − s2)` — one minus the product of the two *death* probabilities `contingency.go:160`). The exclusive statuses `Only1`/`Only2` multiply one survival by the other's complement. Note that `Living + Dead = 1` identically — a complementarity the engine relies on for non-monotone contingencies (§5.4) and which the DOS golden grid confirms numerically (§6).

4.3 The payment-on-death mid-year summation

The death benefit's expected present value is computed by `PODValueFunc(asOf, discountForYears)` `contingency.go:204`; `PODValue(asOf, rate)` `contingency.go:174` is a thin wrapper that passes a constant-rate discount callback e^{-rt}. The algorithm is a discrete **year-of-death summation** with a **mid-year point-mass** for the death timing:

```
for k := 0; ageNow+k < maxAge; k++ {        // each future year of death  contingency.go:224
    pAliveK  := ConditionalSurvival(ageNow, ageNow+k)
    pAliveK1 := ConditionalSurvival(ageNow, ageNow+k+1)
    pDeath   := pAliveK - pAliveK1               // P(die in year k)  contingency.go:228
    if pDeath <= 0 { continue }
    years := offsetToNow + k + 0.5              // MID-YEAR death timing  contingency.go:234
    sum   += POD * pDeath * discountForYears(years)
}
return interest.Round2(sum)                     // DOS round-half-down  contingency.go:238
```

Walking through it: the probability that death falls in year `k` is the drop in conditional survival across that year, `P(alive at k) − P(alive at k+1)`. The benefit for that year is discounted not to the start or end of the year but to its **midpoint**, `k + 0.5` (plus any offset when `asOf ≠ Now` `contingency.go:221`). Summing `POD × P(die in year k) × discount(k + 0.5)` over all future years gives the expected discounted benefit. The discount is a *callback*, which is what lets the variable-rate path (§5.1) discount each death year through its piecewise schedule rather than a single rate. The result is rounded with `interest.Round2` (DOS round-half-down).

> **Why mid-year.** A year-of-death summation has to place the death somewhere within the year. Start-of-year over-values the benefit (deaths discount too little); end-of-year under-values it. The mid-year point-mass is the convention DOS used — confirmed by elimination against the real program (§6), where every other within-year convention missed the DOS POD figure by dollars to a thousand dollars while mid-year landed within a cent.

5 Integration into present value

The overlay is a clean symmetry that mirrors the recovered DOS contract: the survival weight and the POD term are *added* on the forward pass and *removed* on the backward pass. `Calculate` `presentvalue/calc.go:300` runs the two-life guard, then dispatches to `forwardOnly`, `forwardVariableRate`, or `solveUnknownPOD`.

5.1 Forward weighting

A lump value is multiplied by its survival probability in `forwardOnly` `presentvalue/calc.go:498` (`PRESVALU.pas:313`). A periodic stream routes through `periodicWithActuarial` `presentvalue/calc.go:587`, which forces exact period-by-period summation (the closed-form geometric series cannot carry a per-period weight) and multiplies each payment by `LifeProb(t)` at its own date `calc.go:642` (`PRESVALU.pas:427`). The POD value is added to the grand total `presentvalue/calc.go:560` (`PRESVALU.pas:896`). Under a variable rate the same overlay lives in `forwardVariableRate` `variablerate.go:273`: lump weight at `variablerate.go:312`, per-payment weight in `vrPeriodicValue` `variablerate.go:159`, and the POD via `PODValueFunc` with a schedule-aware discount callback `variablerate.go:359`.

5.2 The two-life guard

Because `survivalProb2` silently treats a missing person 2 as immortal (§4.1), any row using a `RequiresSecondLife` contingency without a usable second table + DOB is rejected by name up front — `checkSecondLifeProvided` `presentvalue/calc.go:387`, called from both `Calculate` and the API. The error names the offending row and contingency.

5.3 Backward symmetry & the date-solve prohibition

Backward solves invert the same weighted factor. `computeKnownRowSum` adds the POD value into the known subtotal so it is peeled from the target `presentvalue/backward.go:635` (`PRESVALU.pas:1085`), and amount-solves divide the survival probability back out (erroring when the probability falls below `teeny` — a payment beyond the table horizon) `backward.go:680`. A life-contingent *date* solve is rejected outright `presentvalue/backward.go:724` (`PRESVALU.pas:101,340`, `no_time_with_life`) because the survival probability itself depends on the unknown date, making the equation non-invertible.

The unknown-POD solve `solveUnknownPOD` `presentvalue/calc.go:419` exploits linearity in the benefit amount: it probes `POD = 0` and a large value, divides to get the unit POD value, then solves `POD = (target − cash-flow value) / unit` `calc.go:441` — the same trick the variable-rate `solveVariableRatePOD` uses `variablerate.go:495`.

5.4 The per-payment installments column

DOS's PVL table (`pvltable.pas` `PrintNextPayment`) prints one row per payment date, each with its own probability and an "if paid" value. The Go engine reproduces this: `PeriodicPayment` carries an `Installments []PeriodicInstallment` slice `presentvalue/types.go:75` / `84` — per payment the date, `IfPaid` (discounted but *not* survival-weighted — DOS `ifpd`), `Prob` (`LifeProb(t)`, recovered in DOS as `v/ifpd`), and `Value =`

`IfPaid·Prob` (DOS v). It is built inside `periodicWithActuarial` `calc.go:648` and the variable-rate `vrPeriodicValue`, and surfaced via the API as `PVInstallmentResp` `api/handlers.go:468`.

> **Why the loop never early-breaks.** DOS's exact summation breaks when a term falls below `teeny`. The Go loop deliberately does *not* early-break `calc.go:655`, because `Dead` / `Only1` / `Only2` / `Either` probabilities are **non-monotone** — near zero for a young insured, then growing — so an early break would zero out their value and violate the `Living + Dead = non-contingent` complementarity. The matching DOS handling is the JJM special case (`PRESVALU.pas:551`), discussed in the DOS companion. A regression test (`presentvalue/actuarial_installments_test.go`) locks in that each installment's `Prob` equals `LifeProb(date)`, that `Value = IfPaid·Prob`, and that the installments reconcile to the row value.

6 Validation

This engine is no longer validated against textbook mathematics alone. Three independent checks now anchor it:

- **Real DOS, to the cent.** Driving the original `PerSense.exe` under headless DOSBox on the recovered 1988 tables, the Go engine matches the real program on a four-case golden grid — single-life `Living`, single-life `Dead`, two-life `Both-Living`, and `Living + POD 100k` — each to the cent. (Example: DOS Living Sum Value `104,258.31` vs Go `104,258.3065`; relErr 3.4e-8.) Recorded in `docs/dos_actuarial_golden.md` and guarded by `presentvalue/dos_actuarial_golden_test.go` (`TestDOSActuarialGolden`, opt-in via `PERSENSE_GOLDEN=1`).
- **Mid-year POD convention, confirmed by elimination.** Recomputing case 4's POD under each within-year death-timing convention against the same DOS figure, only the mid-year (k+0.5) point-mass landed within a cent; start-of-year, end-of-year, and the continuous UDD integral each missed by dollars to a thousand dollars (`docs/dos_actuarial_golden.md` §"Case 4"). The mid-year convention §4.3 is therefore DOS-faithful, not an arbitrary choice.
- **actuarialmath, ~500 randomized cases.** `TestActuarialFuzzVsThirdParty` (`actuarial/thirdparty_fuzz_test.go`) throws ~500 randomly drawn life-contingency queries at the engine and diffs each against the live open-source `actuarialmath` library; `TestActuarialLiveThirdPartyOracle` walks a fixed grid. Both skip cleanly when python3 / actuarialmath is absent.

What is *not* yet a full DOS sweep: the golden grid is four hand-captured cases, not the exhaustive differential cube the other engines enjoy, because the `ACTUARY` unit *source* remains missing (a link-against-units oracle cannot be built). The recovered tables plus the runnable binary make a broader black-box DOSBox harness feasible — see `docs/actuarial_dosbox_oracle_plan.md` and `docs/actuarial_oracle_blocked.md`.

7 Appendix: truth table, data flow & file:line index

Code	Contingency	Lives	LifeProb
0	NotContingent	none	1
1	Living	#1	s1
2	Dead	#1	1 − s1
3	Only1Living	both	s1·(1 − s2)
4	Only2Living	both	(1 − s1)·s2
5	EitherLiving	both	1 − (1 − s1)(1 − s2) — last-survivor
6	BothLiving	both	s1·s2 — joint-life

s_i = ConditionalSurvival(age$_i$@Now, age$_i$@date) = l_i(age@date)/l_i(age@Now), conditional on being alive at the valuation date.

```
API (PVActuarialReq) → buildActuarialConfig → ActuarialConfig     api/handlers.go:1547 / contingency.go:86
       {Table1/2 via ParseJSON, DOB1/2, Now, POD | PODUnknown}
            |
            v   table math (UDD interpolation)
   SurvivalProb → ConditionalSurvival  t·p·x                      table.go:44 / 73
            |
            v   LifeProb(date, contingency) — combine s1,s2        contingency.go:142
   +--------+--------------------------------+
   v fixed rate                                v variable rate
   forwardOnly                   calc.go:467   forwardVariableRate     variablerate.go:273
     lump:     val *= LifeProb       :498        lump:     *= LifeProb      :312
     periodic: periodicWithActuarial :587        periodic: vrPeriodicValue  :159
        per-payment ifpd·prob → Installments[]      per-payment → Installments[]
     POD: += PODValue(asOf, rate)    :560        POD: += PODValueFunc(schedule cb) :359
            |
            v   backward: peel POD, invert weighted factor, prohibit date-solve
   computeKnownRowSum +POD backward.go:635 · solveUnknownPOD calc.go:419 · date-solve reject backward.go:724
            |
            v   PeriodicPayment.{Val, Prob(avg), Installments[]} → PVInstallmentResp
                                       types.go:75 / api/handlers.go:468
```

Symbol	Reference
Contingency consts / RequiresSecondLife / labels	`contingency.go:13 / 28 / 38`
ActuarialConfig	`contingency.go:86`
survivalProb1 / survivalProb2	`contingency.go:114 / 125`
LifeProb / PODValue / PODValueFunc	`contingency.go:142 / 174 / 204`
SurvivalProb (UDD) / ConditionalSurvival / LifeExpectancy	`table.go:44 / 73 / 88`
NewLifeTableFromQx / FromLx / ParseCSV / ParseJSON	`table.go:104 / 119 / 139 / 211`
Recovered 1988 tables (Male/Female qx + ctors)	`persense1988.go:24 / 46 / 66 / 71`
Forward lump / periodic / POD add	`presentvalue/calc.go:498 / 642 / 560`
Two-life guard / unknown-POD solve	`presentvalue/calc.go:387 / 419`
Variable-rate overlay (lump / periodic / POD)	`presentvalue/variablerate.go:312 / 159 / 359`
Backward POD peel / amount divide / date-solve reject	`presentvalue/backward.go:635 / 680 / 724`
Installments type / API response	`presentvalue/types.go:75 · api/handlers.go:468`
DOS golden validation / fuzz oracle	`presentvalue/dos_actuarial_golden_test.go · actuarial/thirdparty_fuzz_test.go`

Per%Sense Go port. Citations resolve under `internal/`. For the DOS integration contract and provenance (what is present in the DOS source vs what is missing), see `legacy/code_docs/actuarial_dos_walkthrough.pdf`. This feature overlays the present-value engine — see the present-value Go walkthrough first.